

Rapport de TP

TP2 — Construire un mini Data Lake avec MinIO, Python et PySpark

Réponses aux questions, captures commentées et choix techniques

Binôme : Nour REKIK — Bouchra TYAL

30 septembre 2026

Sommaire

1. Contexte et architecture
2. Environnement de travail
3. Création et alimentation du Data Lake
4. Les images produits
5. Sécurité : utilisateur IAM et policies
6. Vérification des permissions
7. Exploration de MinIO avec Python
8. Exploration des sources et anomalies
9. RAW → PROCESSED : règles de nettoyage
10. Standardisation en Parquet
11. Enrichissement et zone CURATED
12. Comportement de Spark : part-* et _SUCCESS
13. Réexécution et idempotence
14. Conclusion

1. Contexte et architecture

L'entreprise dispose de données hétérogènes : ventes (CSV), clients (JSON), produits (XML), magasins (Excel) et images de produits (JPG). L'objectif est de les organiser dans un Data Lake MinIO, compatible S3, puis de construire avec Python et PySpark un pipeline qui les nettoie, les enrichit et produit des indicateurs métier.

Le bucket **tp2** est organisé en trois zones :

Zone	Rôle	Contenu
RAW	Données sources, dans leur état d'origine, jamais modifiées	ventes.csv, clients.json, produits.xml, magasins.xlsx, 8 images
PROCESSED	Données contrôlées, nettoyées, typées et standardisées	ventes, clients, produits, magasins, ventes_enrichies (Parquet)
CURATED	Données orientées métier, prêtes à l'analyse	ca_par_ville, ca_par_categorie (Parquet)

Le code est réparti en modules : `config.py` (connexion), `spark_utils.py` (session Spark, lecture des sources, écriture Parquet), `test_permissions.py`, `explore_minio.py`, `explore_sources.py` et `pipeline.py`.

2. Environnement de travail

Le projet est placé dans un répertoire indépendant du TP1 (sous macOS : `~/Desktop/datalake/tp2`, équivalent du `D:\datalake\tp2` de l'énoncé). Un environnement virtuel `.venv` a été créé et activé, puis les bibliothèques nécessaires ont été installées et figées dans `requirements.txt`.

Bibliothèque	Utilité
boto3	Accès à MinIO par l'API S3 (listing, lecture, écriture, métadonnées)
pyspark 4.2	Traitements distribués ; lecture native CSV, JSON et XML ; écriture Parquet
pandas + openpyxl	Lecture du fichier Excel, que Spark ne sait pas lire nativement
pyarrow	Support du format Parquet

Le connecteur `hadoop-aws` (protocole `s3a://`) est chargé automatiquement par Spark, dans la version correspondant au Hadoop embarqué dans PySpark (3.5.0).

Question — Pourquoi ne doit-on pas inclure le dossier `.venv` dans le rendu du TP ?

Le dossier `.venv` est volumineux et spécifique à la machine qui l'a créé : il contient des binaires compilés pour un système et une architecture donnés (ici macOS ARM), des chemins absolus et une version précise de Python. Copié sur un autre poste, il ne fonctionnerait pas. Il n'apporte donc rien au correcteur. Le fichier `requirements.txt` suffit : `pip install -r requirements.txt` recrée un environnement équivalent sur n'importe quelle machine.

3. Création et alimentation du Data Lake

Le bucket **tp2** a été créé avec le client `mc`, puis chaque source a été déposée sous son propre préfixe dans la zone RAW, sans aucune modification :

```
mc mb myaistor/tp2
mc cp data/ventes.csv myaistor/tp2/raw/ventes/ventes.csv
mc cp data/clients.json myaistor/tp2/raw/clients/clients.json
mc cp data/produits.xml myaistor/tp2/raw/produits/produits.xml
mc cp data/magasins.xlsx myaistor/tp2/raw/magasins/magasins.xlsx
```

```
mc cp --recursive data/images_produits/ myaistor/tp2/raw/images_produits/
```

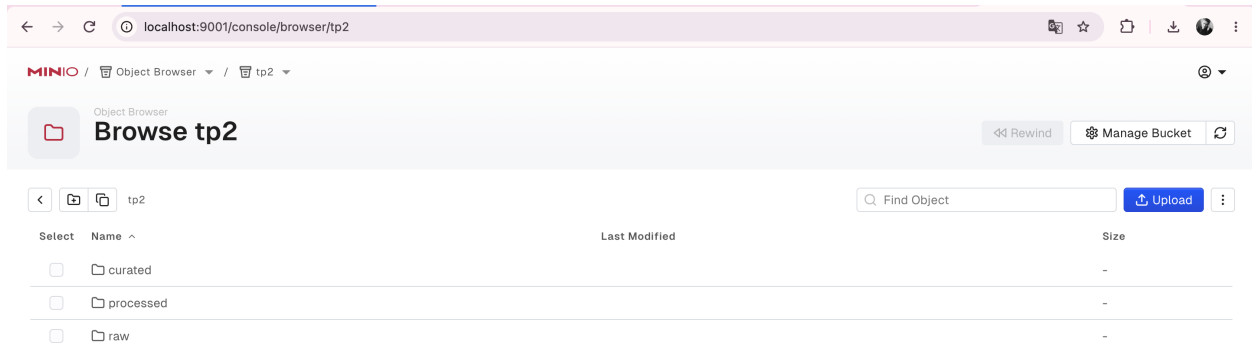


Figure 1 — Data Lake final : bucket tp2 avec ses trois zones

Commentaire. Après exécution du pipeline, le bucket contient les trois zones logiques **raw**, **processed** et **curated**. Ces « dossiers » ne sont pas de vrais répertoires : ce sont des préfixes de clés. Les préfixes processed/ et curated/ n'existaient pas avant la première exécution ; ils sont apparus dès que le pipeline y a écrit des objets.

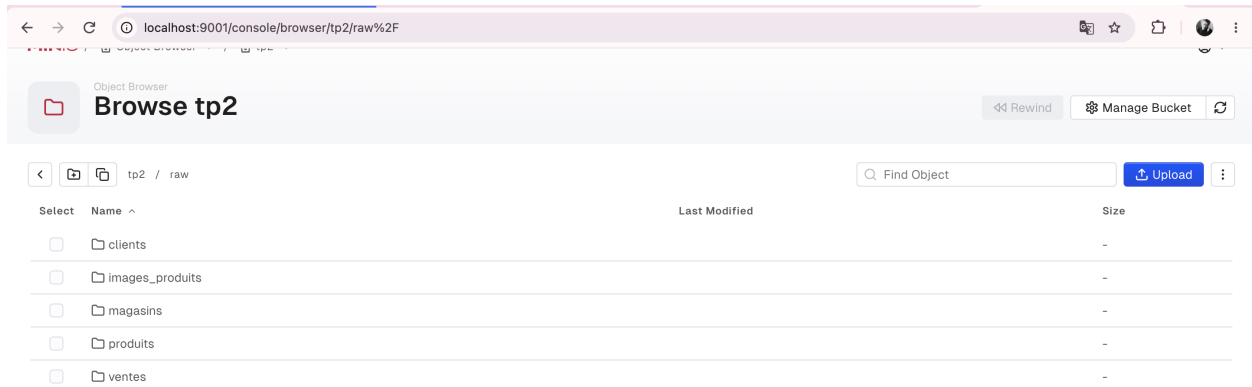


Figure 2 — Contenu de la zone RAW : une source par préfixe

Commentaire. La zone RAW regroupe les cinq sources, chacune sous son préfixe : données structurées (ventes en CSV, magasins en Excel), semi-structurées (clients en JSON, produits en XML) et non structurées (images_produits). Les fichiers sont conservés exactement tels qu'ils ont été fournis, anomalies comprises.

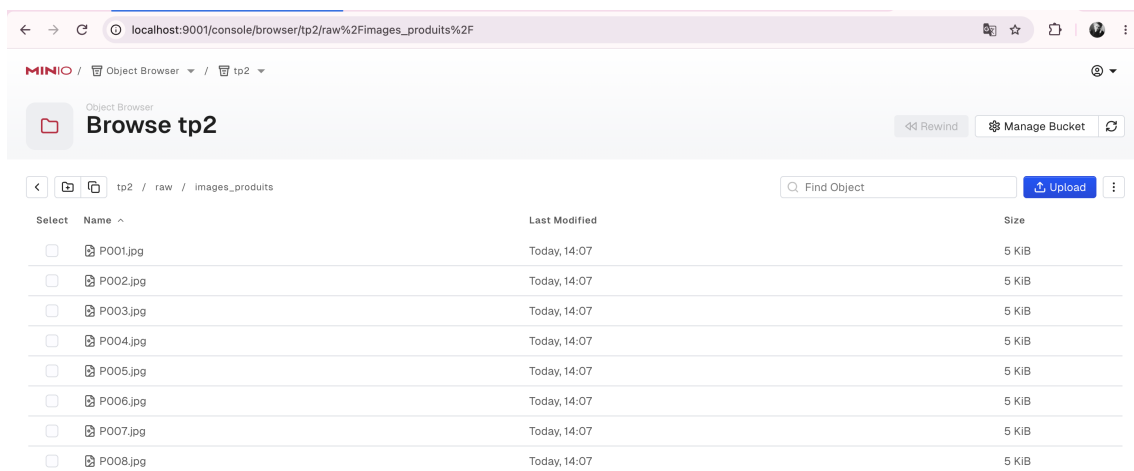


Figure 3 — Les 8 images produits dans raw/images_produits/

Commentaire. Les images P001.jpg à P008.jpg, d'environ 5 Ko chacune, illustrent la présence de données non structurées dans le Data Lake.

Vocabulaire S3

Notion	Définition	Exemple
Bucket	Conteneur de premier niveau	tp2
Objet	Donnée stockée : contenu + métadonnées (taille, type, date, ETag)	le fichier des ventes
Clé (key)	Identifiant complet et unique d'un objet dans le bucket	raw/ventes/ventes.csv
Préfixe	Début commun de plusieurs clés, utilisé pour les regrouper	raw/ ou raw/ventes/

4. Les images produits

Les images sont stockées dans `raw/images_produits/` et ne sont pas traitées : elles ne sont copiées ni dans `PROCESSED` ni dans `CURATED`.

Question — Les images restent uniquement dans RAW à la fin du pipeline. Est-ce une anomalie ?

Non. Un Data Lake a justement vocation à stocker tous les types de données (structurées, semi-structurées et non structurées), et une donnée présente dans RAW n'est pas obligatoirement consommée par tous les pipelines. Notre pipeline produit des indicateurs de ventes (chiffre d'affaires par ville et par catégorie) : les images n'apportent aucune information à ces calculs. Les recopier dans `PROCESSED` ou `CURATED` sans transformation dupliquerait les données sans valeur ajoutée et brouillerait le rôle des zones. Elles restent disponibles dans RAW, dans leur format d'origine, pour un usage futur : catalogue en ligne, vision par ordinateur, extraction de caractéristiques, etc.

5. Sécurité : utilisateur IAM et policies

Un utilisateur dédié à l'application de traitement, **pipeline-tp2**, a été créé, avec une Access Key propre. Ses droits doivent respecter le tableau suivant :

Action	RAW	PROCESSED	CURATED
Lister	✓	✓	✓
Lire	✓	✓	✓
Écrire	✗	✓	✓
Supprimer	✗	✗	✗

Ces permissions sont construites avec **deux policies** complémentaires :

Policy	Actions autorisées	Ressources
tp2-readonly	s3:ListBucket, s3:GetObject	arn:aws:s3:::tp2 et arn:aws:s3:::tp2/*
tp2-write-transformed	s3:PutObject	arn:aws:s3:::tp2/processed/* et arn:aws:s3:::tp2/curated/*

Aucune des deux n'accorde `s3:DeleteObject` : la suppression est donc refusée partout. Séparer lecture et écriture permet de réutiliser `tp2-readonly` seule pour un profil de consultation, par exemple un analyste.

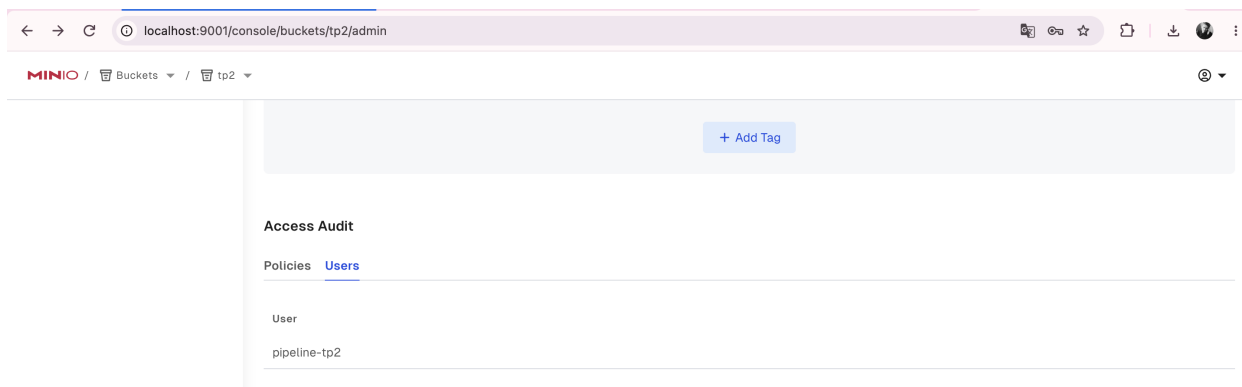


Figure 4 — Utilisateur IAM pipeline-tp2 ayant accès au bucket tp2

Commentaire. L'onglet Access Audit du bucket tp2 montre que l'utilisateur **pipeline-tp2**, créé pour le pipeline, est la seule identité applicative disposant d'un accès à ce bucket.

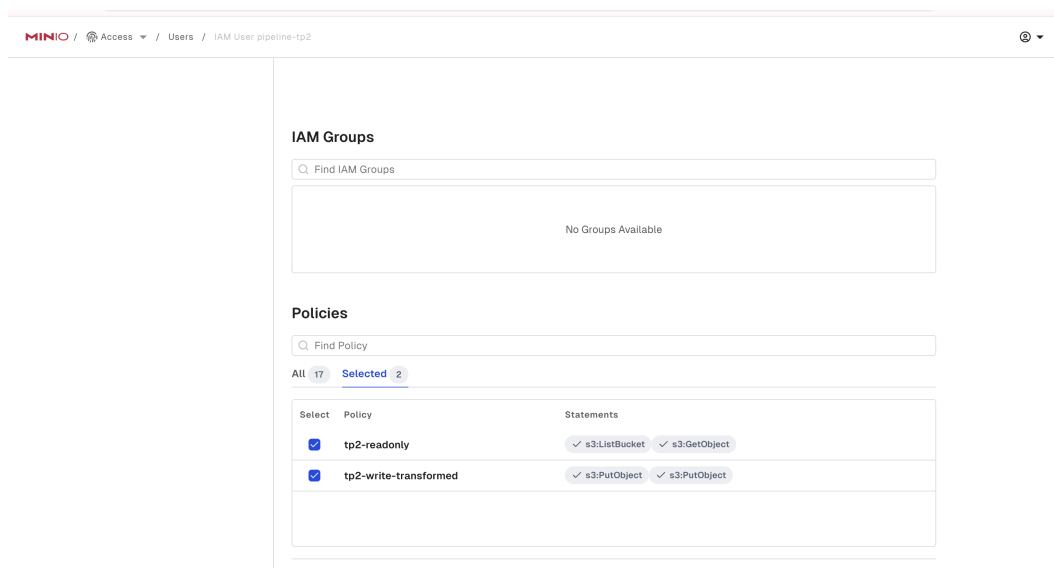


Figure 5 — Politiques associées à l'utilisateur pipeline-tp2

Commentaire. Les deux politiques **tp2-readonly** (s3:ListBucket, s3:GetObject) et **tp2-write-transformed** (s3:PutObject, une fois pour processed et une fois pour curated) sont bien attachées à l'utilisateur. Aucune action de suppression n'apparaît.

Credentials

L'Access Key identifie l'application auprès de MinIO ; la Secret Key sert à signer (authentifier) ses requêtes. La Secret Key n'apparaît jamais dans le code : config.py lit les identifiants d'abord dans des variables d'environnement (MINIO_ACCESS_KEY, MINIO_SECRET_KEY), sinon dans un fichier credentials.json placé à la racine du projet et exclu du rendu. Elle n'apparaît pas non plus dans les captures de ce rapport.

6. Vérification des permissions

Plutôt que de se contenter d'observer les politiques dans l'interface, le script test_permissions.py tente réellement chaque opération avec les credentials de pipeline-tp2 et compare le résultat obtenu au résultat attendu.

```

(.venv) rekiknour@KHALESSI tp2 % touch credentials.json && open -e credentials.json
(.venv) rekiknour@KHALESSI tp2 % python src/test_permissions.py
[--- Test des permissions sur le bucket tp2 ---]

Lister le bucket          -> AUTORISÉE (attendu : AUTORISÉE) OK
Lecture dans RAW         -> AUTORISÉE (attendu : AUTORISÉE) OK
Écriture dans RAW        -> REFUSÉE (attendu : REFUSÉE) OK
Écriture dans PROCESSED  -> AUTORISÉE (attendu : AUTORISÉE) OK
Écriture dans CURATED    -> AUTORISÉE (attendu : AUTORISÉE) OK
Suppression dans RAW     -> REFUSÉE (attendu : REFUSÉE) OK
Suppression dans PROCESSED -> REFUSÉE (attendu : REFUSÉE) OK

```

Figure 6 — Exécution de test_permissions.py

Commentaire. Les 7 tests sont conformes : le listing et la lecture sont autorisés ; l'écriture est refusée dans RAW et autorisée dans PROCESSED et CURATED ; la suppression est refusée dans RAW comme dans PROCESSED. Les droits appliqués correspondent exactement au tableau de l'énoncé.

Question — Lorsqu'une tentative d'écriture dans RAW est refusée, quel mécanisme est responsable de ce refus ?

C'est le **moteur IAM de MinIO**, qui évalue les politiques. Chaque requête S3 est signée avec l'Access Key ; MinIO identifie l'utilisateur pipeline-tp2, rassemble les politiques qui lui sont attachées et cherche une règle Allow couvrant l'action demandée (s3:PutObject) sur la ressource visée (arn:aws:s3:::tp2/raw/*). Aucune n'existe. Or le modèle IAM repose sur le **refus par défaut** : tout ce qui n'est pas explicitement autorisé est interdit. MinIO rejette donc la requête avec une erreur 403 AccessDenied, que boto3 remonte sous forme de ClientError. Ce n'est ni le code Python ni le système de fichiers qui bloque l'écriture, mais bien le serveur de stockage.

7. Exploration de MinIO avec Python

Le programme explore_minio.py communique avec MinIO par son API S3 (boto3) : il se connecte, liste tous les objets du bucket tp2 (avec pagination), les regroupe par zone et par source, lit les métadonnées de plusieurs objets sans les télécharger (head_object) et vérifie la lecture de raw/ventes/ventes.csv. Il n'utilise aucun chemin local.

```

(.venv) rekiknour@KHALESSI tp2 % python src/explore_minio.py
=== Bucket 'tp2' : 12 objet(s) ===

[RAW] 12 objet(s)
  clients          1 objet(s)      1.2 Ko
  images_produits  8 objet(s)      38.3 Ko
  magasins         1 objet(s)      3.8 Ko
  produits         1 objet(s)      1.6 Ko
  ventes           1 objet(s)      2.8 Ko

[PROCESSED] 0 objet(s)

[CURATED] 0 objet(s)

=== Métadonnées de quelques objets RAW ===
- raw/ventes/ventes.csv
  taille      : 2887 octets
  type        : text/csv
  modifié le  : 2026-09-30 12:07:45+00:00
  ETag        : "0c2fc565fc41c353f3f1760a07880c91"
- raw/produits/produits.xml
  taille      : 1596 octets
  type        : text/xml
  modifié le  : 2026-09-30 12:07:46+00:00
  ETag        : "2fc81eb3a0158efc95bb79af43e3b85e"
- raw/magasins/magasins.xlsx
  taille      : 3855 octets
  type        : application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
  modifié le  : 2026-09-30 12:07:46+00:00
  ETag        : "3a47e9a331f04d6edeb9463d87b30acc"
- raw/images_produits/p001.jpg
  taille      : 4855 octets
  type        : image/jpeg
  modifié le  : 2026-09-30 12:07:46+00:00
  ETag        : "5f0ca7d67dd59f11a13eb29d62db3d09"

=== Aperçu de raw/ventes/ventes.csv ===
id_vente,date_vente,id_client,id_produit,id_magasin,quantite
V001,2026-03-23,C002,P001,M006,3.0
V002,2026-02-01,C004,P003,M006,1.0
V003,2026-03-28,C009,P002,M005,4.0
V004,2026-01-05,C001,P002,M002,2.0
... (81 lignes de données)
(.venv) rekiknour@KHALESSI tp2 %

```

Figure 7 — Exécution de explore_minio.py (avant le pipeline)

Commentaire. Le bucket contient 12 objets, tous dans RAW : 1 fichier par source et 8 images. PROCESSED et CURATED sont encore vides. Pour chaque objet, MinIO renvoie la taille, le type MIME (text/csv, text/xml, application/vnd...sheet pour l'Excel, image/jpeg), la date de modification

et l'ETag, qui sert d'empreinte du contenu. L'aperçu de ventes.csv confirme que la lecture dans RAW fonctionne.

Question — Quelle différence faites-vous entre D:\datalake\tp2 et tp2/raw/ventes/ventes.csv ?

D:\datalake\tp2 (chez nous ~/Desktop/datalake/tp2) est un **chemin local** dans le système de fichiers d'un ordinateur : c'est le répertoire du projet, qui contient le code et l'environnement. Il n'existe que sur ce poste. tp2/raw/ventes/ventes.csv désigne un **objet du stockage MinIO** : tp2 est le bucket et raw/ventes/ventes.csv la clé de l'objet. On y accède par le réseau, via l'API S3, depuis n'importe quelle machine disposant des droits. C'est pourquoi les programmes lisent les données par leur bucket et leur clé, et jamais par un chemin local : le pipeline peut ainsi s'exécuter sur un autre poste ou sur un cluster sans modification.

8. Exploration des sources et anomalies

Avant toute transformation, `explore_sources.py` profile chaque source avec PySpark, sans rien corriger : schéma, nombre d'enregistrements, valeurs manquantes (nulle ou vides), doublons et valeurs suspectes. Chaque format est lu avec la méthode adaptée :

Source	Format	Méthode de lecture
Ventes	CSV	Lecteur CSV natif de Spark, toutes colonnes en texte (sans inférence) pour contrôler le typage
Clients	JSON	Lecteur JSON natif avec l'option <code>multiLine</code> (tableau réparti sur plusieurs lignes)
Produits	XML	Lecteur XML natif de Spark 4, avec <code>rowTag</code> = produit
Magasins	Excel	Non supporté par Spark : objet lu via l'API S3, parsé avec <code>pandas/openpyxl</code> , puis converti en <code>DataFrame</code> Spark

Principales anomalies détectées

Source	Lignes	Anomalies
Ventes	81	En-tête précédé d'un BOM UTF-8 ; vente V010 en double ; date invalide 2026-02-30 (V006) ; date absente (V046) ; quantité négative -2 (V012), nulle (V053), absente (V018) ; quantités stockées en décimal (3.0) ; références inexistantes : client C999 (V024), produit P999 (V032), magasin M999 (V040)
Clients	11	Client C010 en double ; villes de casse et d'espacement incohérents (PARIS, paris, « Nantes », NANTES) ; ville vide (C007) ; segment absent (C010)
Produits	10	Catégorie vide (P010) ; aucun prix invalide
Magasins	6	Villes à standardiser (« Rennes » avec espaces, LILLE en majuscules)

9. RAW → PROCESSED : règles de nettoyage

Principe retenu : **ne pas supprimer une ligne dès qu'une anomalie apparaît**. Chaque anomalie est tracée dans une colonne `anomalies`, et une vente n'est écartée des analyses que si elle rend le calcul du montant impossible.

Règle	Traitement
Typage	Dates converties avec <code>try_to_date</code> (une date invalide devient null au lieu de faire échouer le job ; la valeur d'origine est conservée) ; quantités converties en entiers (3.0 → 3) ; prix en double
BOM	Retiré des noms de colonnes
Standardisation	Identifiants sans espaces et en majuscules ; villes nettoyées (trim + majuscule initiale : « PARIS » → Paris, LILLE → Lille) ; ville vide → null ; segment absent → « Non renseigné » ; catégorie vide → « Non renseignée »
Doublons	Ventes dédoublonnées sur <code>id_vente</code> (V010), clients sur <code>id_client</code> (C010)

Cas	Décision	Justification
Date invalide ou absente (V006, V046)	Conservée	La date n'intervient pas dans les indicateurs ; le chiffre d'affaires est réel
Client inconnu C999 (V024)	Conservée (« Client inconnu »)	La vente a eu lieu dans un magasin connu ; la supprimer sous-estimerait le CA
Magasin inconnu M999 (V040)	Conservée (ville « Inconnue »)	Même raison ; elle est isolée dans une ligne dédiée de l'analyse par ville

Cas	Décision	Justification
Produit inconnu P999 (V032)	Écartée	Sans produit, pas de prix : montant incalculable
Quantité absente (V018)	Écartée	Montant incalculable
Quantité nulle (V053)	Écartée	Aucune vente effective
Quantité négative (V012)	Écartée	Probable retour, mais rien ne permet de le confirmer ni de le rattacher à une vente d'origine

Résultat : 80 ventes dans processed/ventes (81 moins le doublon), dont **76 exploitables** ; les 4 ventes écartées restent consultables avec le motif de leur rejet.

10. Standardisation en Parquet

Toutes les données de PROCESSED (ventes, clients, produits, magasins, ventes_enrichies) et de CURATED sont enregistrées au format Parquet.

Question — Les sources étaient en CSV, JSON, XML et Excel. Pourquoi standardiser les données traitées en Parquet ?

- **Un format unique** pour toutes les étapes suivantes : un seul mode de lecture, quel que soit le format d'origine.
- **Schéma et types conservés** dans le fichier : une date reste une date et une quantité un entier, sans réinférence ni erreur de conversion à chaque lecture (contrairement au CSV, où tout est texte).
- **Stockage en colonnes** : Spark ne lit que les colonnes utiles à une requête, ce qui est beaucoup plus rapide sur de gros volumes.
- **Compression efficace** (Snappy) : moins d'espace de stockage et moins de transfert réseau vers MinIO.
- **Format ouvert, standard du Big Data**, lisible par Spark, pandas, DuckDB, Trino, etc.
- Le fichier Excel, non lisible nativement par Spark, devient une source comme les autres une fois converti.

11. Enrichissement et zone CURATED

Les ventes sont reliées aux référentiels par trois clés : `id_client` → clients, `id_produit` → produits (qui apporte la catégorie et le prix) et `id_magasin` → magasins (qui apporte la ville et la région). Les jointures sont de type *left*, pour ne perdre aucune vente. Le montant est calculé par **montant = quantite × prix_unitaire**, et le résultat est stocké dans processed/ventes_enrichies.

Les agrégations **repartent de PROCESSED** (ventes_enrichies) et jamais des fichiers RAW. L'analyse par ville utilise la ville du magasin, c'est-à-dire le lieu où la vente a été réalisée.

```

magasins : 6 ligne(s) brutes

[2/4] RAW -> PROCESSED (nettoyage + Parquet)
ventes : 80 ligne(s) après nettoyage
-> s3a://tp2/processed/ventes/ (1 fichier(s) part + _SUCCESS)
clients : 10 ligne(s) après nettoyage
-> s3a://tp2/processed/clients/ (1 fichier(s) part + _SUCCESS)
produits : 10 ligne(s) après nettoyage
-> s3a://tp2/processed/produits/ (1 fichier(s) part + _SUCCESS)
magasins : 6 ligne(s) après nettoyage
-> s3a://tp2/processed/magasins/ (1 fichier(s) part + _SUCCESS)

[3/4] Enrichissement des ventes (lecture depuis PROCESSED)
ventes exploitables : 76 | écartées : 4
+-----+-----+-----+-----+-----+
|id_vente|id_produit|quantite|prix_unitaire|anomalies|
+-----+-----+-----+-----+-----+
|V053|P004|0|59.0|quantite nulle|
|V012|P010|-2|21.5|quantite negative|
|V018|P003|NULL|49.9|quantite absente|
|V032|P999|2|NULL|produit inconnu|
+-----+-----+-----+-----+-----+

-> s3a://tp2/processed/ventes_enrichies/ (1 fichier(s) part + _SUCCESS)

[4/4] PROCESSED -> CURATED (agrégations)
-> s3a://tp2/curated/ca_par_ville/ (1 fichier(s) part + _SUCCESS)
-> s3a://tp2/curated/ca_par_categorie/ (1 fichier(s) part + _SUCCESS)

=== Contrôle des résultats CURATED (relus depuis MinIO) ===
+-----+-----+-----+-----+
|ville|nb_ventes|quantite_totale|chiffre_affaires|
+-----+-----+-----+-----+
|Lille|15|62|2093.4|
|Paris|16|54|1982.6|
|Lyon|11|47|1876.9|
|Rennes|12|38|1354.25|
|Marseille|13|42|1350.2|
|Nantes|8|25|1049.6|
|Inconnue|1|5|199.5|
+-----+-----+-----+-----+

+-----+-----+-----+-----+
|categorie|nb_ventes|quantite_totale|chiffre_affaires|
+-----+-----+-----+-----+
|Informatique|29|106|3480.8|
|Audio|18|61|3444.3|
|Accessoires|13|53|1940.9|
|Maison|10|34|631.95|
|Non renseignée|6|19|408.5|
+-----+-----+-----+-----+

Pipeline terminé avec succès. (raw/images_produits/ conservées dans RAW uniquement)
(.venv) rekiknour@KHALESSI tp2 %

```

Figure 8 — Exécution complète et réussie du pipeline

Commentaire. Le pipeline enchaîne les quatre étapes sans intervention manuelle : lecture de RAW, écriture des quatre jeux nettoyés dans PROCESSED (80 ventes, 10 clients, 10 produits, 6 magasins), enrichissement (76 ventes exploitables, 4 écartées avec leur motif), puis écriture de ca_par_ville et ca_par_categorie dans CURATED. Les deux tableaux de contrôle sont relus depuis MinIO, ce qui prouve que les résultats sont bien stockés.

Analyse 1 — Activité par ville (curated/ca_par_ville)

Ville	Nombre de ventes	Quantité totale	Chiffre d'affaires (€)
Lille	15	62	2 093,40
Paris	16	54	1 982,60
Lyon	11	47	1 876,90
Rennes	12	38	1 354,25
Marseille	13	42	1 350,20
Nantes	8	25	1 049,60
Inconnue	1	5	199,50

Analyse 2 — Activité par catégorie (curated/ca_par_categorie)

Catégorie	Nombre de ventes	Quantité totale	Chiffre d'affaires (€)
Informatique	29	106	3 480,80
Audio	18	61	3 444,30
Accessoires	13	53	1 940,90
Maison	10	34	631,95
Non renseignée	6	19	408,50

Contrôle de cohérence : le chiffre d'affaires total est identique dans les deux analyses (9 906,45 €) et le nombre total de ventes vaut 76 dans les deux cas. Lille arrive en tête malgré moins de ventes que Paris, grâce à un panier plus élevé ; l'Informatique et l'Audio représentent à elles deux 70 % du chiffre d'affaires.

12. Comportement de Spark : fichiers part-* et _SUCCESS

Dans MinIO, chaque jeu de données Parquet apparaît comme un préfixe contenant plusieurs objets, et non comme un fichier unique :

```
processed/ventes_enrichies/part-00000.snappy.parquet
processed/ventes_enrichies/_SUCCESS
```

Question — Pourquoi Spark produit-il des fichiers part-* ?

Spark est un moteur de calcul **distribué** : un DataFrame est découpé en **partitions**, traitées en parallèle par plusieurs tâches, sur plusieurs cœurs ou plusieurs machines. À l'écriture, chaque tâche écrit sa partition dans son propre fichier part-XXXXX, indépendamment des autres et sans coordination, ce qui permet d'écrire en parallèle. Un jeu de données est donc un dossier (un préfixe) de un ou plusieurs fichiers. Nos volumes étant très faibles, chaque jeu est regroupé en une seule partition (`coalesce(1)`), d'où un seul fichier part-00000.

Question — Que représente _SUCCESS ?

C'est un fichier vide, écrit à la toute fin, une fois que toutes les tâches ont terminé avec succès. Il signale que le jeu de données est **complet et cohérent**. Son absence indique une écriture interrompue ou en cours : un traitement en aval peut s'en servir comme signal de disponibilité avant de lire les données. Les fichiers commençant par « _ » sont ignorés par Spark lors de la lecture.

Question — Quel lien avec le fonctionnement distribué de Spark ?

Un fichier part-* correspond au travail d'une tâche (une partition), et _SUCCESS correspond à la validation globale du job par le driver, une fois que toutes les tâches parallèles ont réussi. Cette organisation permet d'écrire en parallèle, de relire en parallèle (une tâche par fichier) et de ne jamais exposer un résultat partiel comme s'il était terminé.

13. Réexécution et idempotence

Problème rencontré

L'écriture Spark directe vers MinIO (`df.write.parquet("s3a://...")`) repose sur le `FileOutputCommitter` : il écrit d'abord dans un dossier `_temporary/`, puis le **supprime** ; et le mode `overwrite` **supprime** l'ancien résultat. Ces deux opérations exigent `s3:DeleteObject`, refusé à `pipeline-tp2` (section 5). Les deux approches naïves échouent donc : `overwrite` est refusé, et `append` accumulerait des doublons à chaque exécution.

Stratégie adoptée

1. Spark écrit le jeu de données en Parquet dans un dossier temporaire local.
2. Le programme envoie chaque fichier dans MinIO avec `PutObject`, sous une **clé fixe et déterministe** (`<zone>/<jeu>/part-00000.snappy.parquet`), puis `_SUCCESS` en dernier.
3. Le dossier temporaire local est supprimé.

Comme les clés sont toujours les mêmes, une réexécution **écrase** les objets existants : `PutObject` remplace un objet de même clé sans nécessiter le droit de suppression. Par ailleurs, les entrées sont dédoublonnées à chaque exécution et tous les résultats sont recalculés intégralement à partir de RAW, qui n'est jamais modifié.

Vérification

Le pipeline a été exécuté deux fois de suite sans suppression manuelle. Après la seconde exécution, le bucket contient toujours exactement 26 objets (12 dans RAW, 10 dans

PROCESSED, 4 dans CURATED), sans fichier supplémentaire ni dossier _temporary, avec les mêmes nombres de lignes (80 ventes, 76 ventes enrichies) et les mêmes indicateurs. **Le pipeline est donc idempotent.**

Limite : cette stratégie suppose un nombre fixe de fichiers par jeu (ici un). Si le nombre de partitions diminuait d'une exécution à l'autre, d'anciens fichiers part-* pourraient subsister. Sur de gros volumes en production, on utiliserait plutôt un format de table transactionnel (Delta Lake, Apache Iceberg), ou un compte technique disposant du droit de suppression uniquement sur les zones qu'il gère.

14. Conclusion

Ce TP nous a permis de construire un Data Lake complet et sécurisé : cinq sources hétérogènes stockées dans leur format d'origine, des droits d'accès minimaux vérifiés par des tests réels, et un pipeline PySpark qui nettoie, standardise en Parquet, enrichit et agrège les données jusqu'à des indicateurs métier. La contrainte d'absence de droit de suppression nous a obligées à comprendre en détail le mécanisme d'écriture de Spark, et à concevoir une stratégie d'écriture à clés fixes qui rend le pipeline idempotent tout en respectant le principe du moindre privilège.